

MAE 242: Robot Motion Planning

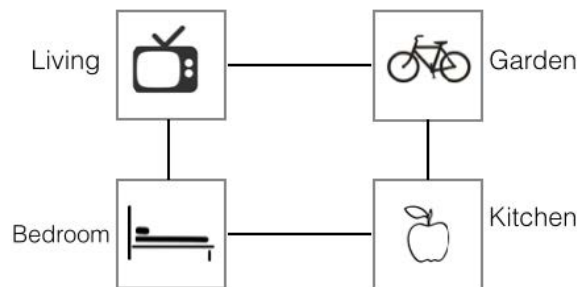
Homework #5, programming homework

Assigned May 22, due June 5

Homework Instructions:

In this programming part, you will implement the Q-learning algorithm, which is a model-free algorithm used to learn an optimal Q -function in a “home-world game”. In the tabular setting, the algorithm maintains the Q -value for all possible state-action pairs. Starting from a random Q -function, the agent continuously collects experiences $(h, c, R(h, c), h')$ and updates its Q -function.

From now on, we will refer to $c = (a.b)$ as “an action” although it really is an action with an object.



Exercises (algorithm implementation):

1. Write a function `tabular_q_learning` that updates a single Q -value given the transition $(h, c, R(h, c), h')$. Use the template provided in the homework files.
2. Note that the Q -learning algorithm does not specify how we should interact in the world so as to learn quickly. It merely updates the values based on the experience collected. If we explore randomly, i.e., always select actions at random, we would most likely not get anywhere. A better option is to exploit what we have already learned, as summarized by current Q -values. We can always act greedily with respect to the current estimates, i.e., take an action $\pi(h) = \arg \max_{c \in C} Q(h, c)$. Of course, early on, these are not necessarily very good actions. For this reason, a typical exploration strategy is to follow a so-called **ϵ -greedy policy**: with probability ϵ take a random action out of C ; with probability $1 - \epsilon$ follow $\arg \max_{c \in C} Q(h, c)$. The value of ϵ here balances exploration vs exploitation. Now you will write a function `epsilon_greedy` that implements the ϵ -greedy exploration policy using the current Q -function. Use the template provided in the homework files.

Exercises (algorithm evaluation):

Now, you will evaluate your learning algorithm for the Home World game. The metric we use to measure an agent's performance is the cumulative discounted reward obtained per episode averaged over the episodes.

The evaluation procedure is as follows. Each experiment (or run) consists of multiple epochs (the number of epochs is `NUM_EPOCHS`). In each epoch:

1. You first train the agent on `NUM_EPIS_TRAIN` episodes, following an ϵ -greedy policy with $\epsilon = \text{TRAINING_EP}$ and updating the Q values.
2. Then, you have a testing phase of running `NUM_EPIS_TEST` episodes of the game, following an ϵ -greedy policy with $\epsilon = \text{TESTING_EP}$, which makes the agent choose the best action according to its current Q -values 95% of the time. At the testing phase of each epoch, you will compute the cumulative discounted reward for each episode and then obtain the average reward over the `NUM_EPIS_TEST` episodes.

Finally, at the end of the experiment, you will get a sequence of data (of size `NUM_EPOCHS`) that represents the testing performance at each epoch.

Note that there is randomness in both the training and testing phase. You will run the experiment `NUM_RUNS` times and then compute the averaged reward performance over `NUM_RUNS` experiments.

Most of these operations are handled by the boilerplate code provided in the `agent_tabular_QL.py` file by functions `run`, `run_epoch` and `main`, but you will need to complete the `run_episode` function.

Questions: Do the following

1. Write a `run_episode` function that takes a boolean argument (whether the episode is a training episode or not) and runs one episode. *Note:* You should implement this function locally first. Make sure you can achieve reasonable performance on the Home World game before submitting your code.
2. Report performance. In your Q -learning algorithm, initialize Q at zero. Set `NUM_RUNS = 10`, `NUM_EPIS_TRAIN = 25`, `NUM_EPIS_TEST = 50`, $\epsilon = 0.5$, `TRAINING_EP = 0.5`, `TESTING_EP = 0.05` and the learning rate $\alpha = 0.1$. What is the number of epochs when the learning algorithm converges? (That is, the testing performance become stable)
3. What is the average episodic rewards of your Q -learning algorithm when it converges?
4. (Impact of ϵ on the convergence of Q -learning algorithm). Which of the behaviors below do you observe from running the algorithm?
 - For very large ϵ (say $\epsilon = 1$) the algorithm converges slower compared to $\epsilon = 0.5$.
 - For very large ϵ (say $\epsilon = 1$) the algorithm converges faster compared to $\epsilon = 0.5$.
 - For very small ϵ (say $\epsilon = 0.000001$) the algorithm converges slower compared to $\epsilon = 0.5$.
 - For very small ϵ (say $\epsilon = 0.000001$) the algorithm converges faster compared to $\epsilon = 0.5$.
5. (Effects of α). Fix the exploration parameter $\epsilon = 0.5$ and do the experiments with different values of the training $\alpha \in [10^{-6}, 1]$. What do you observe?
 - The algorithm converges for all values of α in less than 200 epochs.
 - The algorithm does not converge for all values of α in less than 200 epochs.
 - The smaller the α the faster the convergence.
 - The smaller the α the slower the convergence.

Programming Instructions:

- This assignment requires the use of `tqdm`, `string`, `csv`, `sys` libraries.
- You will fill in the functions `tabular_q_learning`, `epsilon_greedy` provided in `agent_tabular_ql.py` for solving this assignment. You should submit `agent_tabular_ql.py` with your code and comments. Please do not change the other files in this distribution or submit any of our original files other than `agent_tabular_ql.py`.
- The functions modified for this assignment are:

1. `tabular_q_learning(...)`,
2. `epsilon_greedy(...)`,

where we update the the global variable and output actions. We suggest that you review the Q-learning algorithm and the analytical homework before working on this assignment. Please pay attention to the bi-state and action-object definitions. The user defined inputs include discount factor γ , and weight α .

- Your code will be verified for basic technical correctness. Please do not change the names of any provided functions, classes or file names. There will not be any autograder for this part of the assignment.
- Getting Help:
 1. Some sample code has been provided towards the end of the search file to understand the use of functions. Please start early on the coding assignments. If you are stuck on something, please use office hours and Piazza.